Intelligent Test Case Generation with Multi-Framework Code Synthesis

R26-SE-039

Project Proposal Report

ABEYGUNASEKARA D T– IT22303684

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology Sri Lanka

March 2026

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Name	Student ID	Signature
Abeygunasekara D T	IT22303684	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Mrs. Suriya Kumari	14/03/2026	

ABSTRACT

In many software projects, testers still design and write test cases manually by reading acceptance criteria such as Gherkin “Given–When–Then” scenarios. This manual process takes a lot of time, depends heavily on individual experience, and often misses important edge cases, which can lead to lower coverage and hidden defects in both user interfaces and backend logic. Even when tests are automated, QA engineers must usually implement them separately for each framework (for example, Selenium, Cypress, or Playwright), which adds more work and increases the risk of errors in the test code.

This project focuses on building a practical and intelligent bridge between acceptance criteria and executable automated tests. The first part is an Intelligent Test Case Generation module that uses semantic parsing and constraint-based analysis to convert Gherkin-style acceptance criteria into a rich set of test scenarios, including happy paths, boundary value cases, invalid inputs, and other edge conditions. The second part is a Framework Agnostic Test Code Generation module that takes these abstract scenarios and automatically produces type-safe, framework-specific test scripts for web UI and backend testing, following idiomatic patterns and using appropriate locators and assertions. Together, these components aim to reduce the time and effort needed to move from written acceptance criteria to reliable automated tests, while improving coverage and consistency across different test frameworks. The system will be evaluated on sample applications by measuring generation time, code coverage, and the correctness and executability of the generated tests, to show that this approach is both technically feasible and beneficial for real QA workflows.

Keywords: Acceptance criteria, Gherkin, Intelligent test generation, Framework-agnostic test code, Type safety, UI and backend testing

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor and lecturers at the Sri Lanka Institute of Information Technology for their valuable guidance and support during the preparation of this research proposal. Their insights and recommendations have helped shape the direction of this research.

I would also like to thank my project team members for their cooperation and collaboration throughout the project development process. Finally, I would like to acknowledge the academic resources and facilities provided by SLIIT which enabled the successful preparation of this proposal.

TABLE OF CONTENTS

DECLARATION 2

ABSTRACT 3

ACKNOWLEDGEMENT 4

LIST OF TABLES 7

LIST OF FIGURES 8

LIST OF ABBREVIATIONS 9

1. INTRODUCTION 11

1.1 Background and Context 11

1.2 Literature Review 13

1.3 Research Gap 14

2. OBJECTIVE 15

2.1 Main Objective 15

2.2 Specific Objectives 15

3. METHODOLOGY 16

3.1 Research Approach 16

3.2 Technical Methods and Algorithms 16

3.3 Tools and Platforms with Justification 18

3.4 Data Collection and Ethical Considerations 18

3.5 Validation Strategy and Performance Metrics 19

4. HIGH-LEVEL SYSTEM ARCHITECTURE 20

5. USER REQUIREMENTS 21

5.1 Stakeholders 21

5.2 Functional Requirements 21

5.3 Non Functional Requirements 22

6. COMMERCIALIZATION PLAN 23

6.1. Target Market and Customer Persona 23

6.2. Value Proposition 24

6.3. Revenue Model 24

6.4. Cost Estimation 24

6.5. Pricing Strategy 25

6.6. Competitive Advantage 25

6.7. Intellectual Property 25

7. BUDGET AND JUSTIFICATION 27
8. WORK BREAKDOWN STRUCTURE (WBS) 28
9. GANTT CHART 29
10. REFERENCES LIST 30
11. APPENDICES 32

LIST OF TABLES

Table 1: Tools and Platforms Used for System Implementation 21
Table 2: Stakeholder Analysis 29
Table 3: Cost Estimation for System Development 33
Table 4: Pricing Strategy and Subscription Plans 33
Table 5: Budget Allocation for Hardware 35
Table 6: Budget Allocation for Software 35
Table 7: Budget Allocation for Cloud Services 35
Table 8: Budget Allocation for Human Effort 36
Table 9: Total Estimation Budget 36

LIST OF FIGURES

Figure 1: High-Level Architecture of the NEXTGEN QA System 26
Figure 2: System Architecture Diagram for Individual Component 26
Figure 3: Work Breakdown Structure Diagram..... 37
Figure 4: Gantt Chart 38

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
LLM	Large Language Model
QA	Quality Assurance
UI	User Interface
NLP	Natural Language Processing
CI/CD	Continuous Integration / Deployment
GUI	Graphical User Interface
SUT	System Under Test
APT	Application Programming Interface

1. INTRODUCTION

1.1 Background and Context

Modern software development practices rely heavily on Agile methodologies and Behavior Driven Development (BDD) to align business requirements with technical implementation. Gherkin-based acceptance criteria written in Given-When-Then format serve as the primary bridge between product owners and QA engineers, defining the expected behavior of software systems. Automated testing frameworks such as Selenium, Cypress, and Playwright are widely adopted in Continuous Integration and Continuous Deployment (CI/CD) pipelines to validate these requirements through executable test scripts.

However, the process of transforming Gherkin acceptance criteria into comprehensive, executable automated tests remains largely manual and heavily dependent on the individual experience of QA engineers. Engineers must manually interpret each scenario, identify boundary conditions, generate negative test cases, and implement framework-specific test code. This manual process is time-consuming, inconsistent, and frequently results in incomplete test coverage, missed edge cases, and test suites that are tightly coupled to specific automation frameworks.

Studies in software testing have demonstrated that a significant proportion of software defects escape into production due to inadequate test coverage during the development phase. Existing test generation tools primarily rely on template-based mappings or simple keyword matching without any semantic understanding of the acceptance criteria. These tools lack the intelligence to automatically derive boundary values, identify constraint violations, or predict high-risk scenarios based on historical defect patterns.

To address these challenges, the NEXTGEN QA framework proposes an intelligent automated testing system consisting of four main components: Voice/NLP Intelligence, Test Generation and Execution Intelligence, Failure Analysis and Self-Healing Intelligence, and Quality Prediction with Requirement Traceability Matrix (RTM) generation. Within this framework, the Intelligent Test Case Generation component focuses on automatically transforming Gherkin acceptance criteria into comprehensive test scenarios and generating type-safe, idiomatic test code for multiple automation frameworks simultaneously.

Several existing tools such as Cucumber, Spec Flow, and LLM-based code assistants provide partial support for Gherkin to code transformation. However, most of these tools lack intelligent mechanisms for semantic understanding, automatic edge case generation, ML based risk prediction, or framework-agnostic code synthesis. Therefore, there is a need for a more advanced solution that combines NLP-based semantic parsing, machine learning classification, constraint based scenario synthesis, and multi-framework code generation into a single unified pipeline.

This research aligns with Sustainable Development Goal 9 (Industry, Innovation, and Infrastructure) because it supports the development of more efficient, reliable, and intelligent software testing infrastructure. It also aligns with the Software Systems and Technologies research cluster, which focuses on intelligent automation and AI-driven software engineering practices.

1.2 Literature Review

- Automated test case generation from natural language requirements has been an active area of research in software engineering. Research has shown that test cases can be automatically derived from natural language descriptions of software behavior by transforming requirement statements into executable test scenarios using NLP-based parsing pipelines [1].
- Research on semantic parsing for test generation has explored various NLP techniques. One approach, Test Generation with Semantic Role Labeling (TG-SRL), extracts preconditions, actions, and constraints from natural language requirements and combines machine learning with rule-based methods to generate comprehensive test suites [2]. Similarly, systematic studies on AI-driven Behavior Driven Development (BDD) testing highlight that Gherkin based scenario parsing remains a complex problem requiring deeper semantic understanding and improved NLP techniques [3].
- The use of machine learning for edge case prediction and test prioritization has also been widely investigated. Search-based test generation techniques can automatically produce boundary value and equivalence class test cases, significantly improving test coverage compared to manually designed test suites [4]. Complementing this approach, AI-based systems trained on historical defect data can predict high-risk edge cases and negative scenarios, reducing manual effort for QA engineers [5].
- Boundary value analysis and equivalence partitioning remain fundamental techniques in systematic test design. These methods provide a theoretical foundation for deriving comprehensive test scenarios from software specifications and form the basis for constraint-based test synthesis approaches [6].
- Research on multi-framework test code generation has identified significant challenges in producing maintainable and framework-independent test scripts. Studies comparing automated test generation across different web testing frameworks show that framework-specific implementations often lead to high maintenance costs, emphasizing the need for abstract and reusable test representations [7].
- Mutation testing is widely used to evaluate test suite effectiveness. Studies demonstrate that mutation scores provide a reliable indicator of a test suite's ability to detect faults and measure test coverage quality [8].
- Human-in-the-loop approaches in AI-assisted software engineering have also gained attention. Recent research on AI-driven BDD testing shows that combining automated AI generation with human validation and refinement significantly improves the accuracy and reliability of generated test artefacts [3].
- However, most existing approaches treat semantic parsing, edge case prediction, and code generation as separate research areas. There is currently a lack of a unified, end-to-end pipeline that integrates NLP based Gherkin parsing, machine learning-based risk prediction, constraint synthesis, and multi-framework type-safe code generation within a single intelligent system. This research gap motivates the proposed component within the NextGen QA framework.

1.3 Research Gap

Research Value of the Component

Existing test generation tools primarily rely on template-based mappings or simple keyword matching without any semantic understanding of the acceptance criteria. These tools lack the intelligence to automatically derive boundary values, identify constraint violations, or predict high-risk scenarios based on historical defect patterns. Studies in software testing have demonstrated that a significant proportion of software defects escape into production due to inadequate test coverage during the development phase. While tools such as Cucumber, SpecFlow, and LLM-based code assistants provide partial support for Gherkin-to-code transformation, most lack intelligent mechanisms for semantic understanding, automatic edge case generation, ML-based risk prediction, or framework-agnostic code synthesis. Therefore, there is a clear research gap in combining NLP-based semantic parsing, machine learning classification, constraint-based scenario synthesis, and multi-framework code generation into a single unified pipeline.

Product Value

From a product perspective, no commercially available QA tool currently provides a unified pipeline that begins from raw Gherkin acceptance criteria and simultaneously outputs type-safe, idiomatic test scripts for Selenium, Cypress, and Playwright in under 10 seconds. Existing commercial tools such as Testim, mabl, and Katalon Studio offer AI-assisted test creation but treat tests as static scripts and focus mainly on UI-level robustness rather than deeply understanding acceptance criteria. The maintainability challenges of test suites tied to specific locator strategies and frameworks have been well-documented, highlighting the need for a framework-agnostic approach. Current solutions also do not integrate mutation-testing-based coverage gap analysis with automated scenario re-suggestion, leaving QA teams to manually identify missing coverage.

Novelty

Research Value of the Component

This research introduces a novel Human-in-the-Loop Test Generation Architecture that combines AI-driven scenario synthesis with interactive QA refinement, enabling real-time test modification and instant multi-framework code regeneration. The unified pipeline simultaneously merges semantic parsing using NLP, ML-based edge case prediction trained on historical defect data, and constraint-based scenario synthesis — an approach not demonstrated in prior literature. The integration of Random Forest classification for risk-based test prioritization with mutation testing for coverage gap detection and automatic scenario suggestion represents a distinct academic contribution to the field of intelligent test automation.

Product Value

From a product standpoint, the proposed system delivers measurable practical value by generating type-safe, idiomatic test scripts for Selenium, Cypress, and Playwright simultaneously from a single Gherkin input in under 10 seconds. It removes the need for QA engineers to manually re-implement test logic across multiple frameworks, directly reducing engineering hours and improving CI/CD pipeline reliability. The human-in-the-loop review mechanism ensures AI-generated test suggestions are auditable and require explicit QA approval before execution, making the system both responsible and enterprise-ready. Unlike existing tools, this component integrates seamlessly with shared CI/CD infrastructure through QA-approved JSON artifacts stored in a persistent shared database, enabling traceability from acceptance criteria to execute tests.

Technologies and methods	Template-based Gherkin generators	Model- / search-based test generation	General code-gen / LLM tools	Commercial AI testing tools	Proposed Method
Start from Gherkin acceptance criteria	Yes	No	Partial (natural language)	Partial	Yes
Semantic understanding of steps and constraints	Limited (keyword / template)	No	Limited	Limited	Yes
Automatic edge and boundary case generation	No or very limited	Yes (from models / search)	No	No	Yes
ML-based edge case prediction from historical bugs	No	Rare	No	No	Yes
Coverage validation and gap analysis (line/branch, mutation)	No	Sometimes	No	Limited	Yes
Multi-framework support (Selenium, Cypress, Playwright, …)	Usually single-framework	Not the focus	Not specific to test framework	Limited	Yes
Type-safe, framework-idiomatic test code generation	Basic templates only	No	Often non-idiomatic, not type-safe	Not guaranteed	Yes

Unified pipeline from AC → scenarios → executable test code	Partial (steps → skeleton code)	No	No	No	Yes

Table 1.1 Comparison of the existing research methods and proposed methods

2. OBJECTIVE

2.1 Main Objective

To design and develop an intelligent test case generation module that automatically transforms Gherkin acceptance criteria into comprehensive test scenarios and generates framework-agnostic, type-safe test code for Selenium, Cypress, and Playwright automation frameworks within Agile CI/CD pipelines.

2.2 Specific Objectives

1. To implement a semantic parsing engine using Named Entity Recognition (NER) and dependency parsing to extract preconditions, actions, constraints, and expected outcomes from Gherkin Given-When-Then acceptance criteria.
2. To develop a constraint-based scenario synthesis module that automatically applies boundary value analysis and equivalence partitioning to generate comprehensive positive, negative, and edge case test scenarios from extracted Gherkin elements.
3. To build a machine learning model using a Random Forest classifier trained on historical defect data to predict high-risk edge cases and negative test scenarios likely to uncover software faults.
4. To design and implement a framework-agnostic Abstract Test Model that decouples test logic from framework-specific implementation, enabling reusable and maintainable test representations.
5. To develop a multi-framework code generation engine that produces idiomatic, type-safe, executable test scripts for Selenium, Cypress, and Playwright from the Abstract Test Model.
6. To integrate a mutation testing validation module that evaluates the fault detection capability of generated test suites, identifies coverage gaps, and triggers automatic scenario regeneration to improve overall test coverage.
7. To evaluate the performance of the proposed system using metrics such as scenario generation latency, mutation score improvement, branch coverage uplift, and expert assessment of generated test quality compared to manually designed test suites.

3. METHODOLOGY

3.1 Research Approach

- This research adopts a Design Science Research (DSR) approach to design and evaluate an intelligent pipeline for automated test case generation and multi-framework code synthesis. Design Science Research is appropriate because the objective of this study is to create a technological artifact that solves a real-world problem faced by QA engineers in modern Agile software development environments.

- The proposed solution addresses the inefficiency and inconsistency of manual test case design from Gherkin acceptance criteria in Agile CI/CD pipelines. The system will semantically parse Gherkin scenarios, synthesize comprehensive test cases using constraint-based analysis, predict high-risk edge cases using machine learning, and automatically generate type-safe, idiomatic test code for multiple automation frameworks. In addition, the system will validate generated test quality through mutation testing and improve coverage through automatic gap-filling mechanisms.

- The research process consists of the following stages:
  - Collection and preprocessing of Gherkin acceptance criteria from Agile user stories
  - Semantic parsing and structured element extraction using NER and dependency parsing
  - Constraint-based scenario synthesis using boundary value analysis and equivalence partitioning
  - ML-based edge case prediction using Random Forest classifier trained on historical defect data
  - Abstract Test Model construction as a framework-agnostic intermediate representation
  - Multi-framework type-safe test code generation for Selenium, Cypress, and Playwright
  - Mutation testing validation and automatic coverage gap detection and regeneration
  - Experimental evaluation of system performance against manually designed test suites

3.2 Technical Methods and Algorithms

The proposed framework consists of several modules that operate sequentially to transform Gherkin acceptance criteria into comprehensive, type-safe, executable test scripts across multiple automation frameworks.

Gherkin Input Collection & Preprocessing

- Gherkin. feature files containing Given-When-Then acceptance criteria are collected from Agile project repositories via the React Dashboard interface. These files are preprocessed by tokenizing scenario text, removing noise, and structuring each step into its component parts keyword, action phrase, and parameter values ready for semantic analysis.

Semantic Parsing

- Natural Language Processing techniques including Named Entity Recognition (NER) and dependency parsing are applied to each Gherkin step using spaCy and Stanford NLP. These techniques extract structured elements such as preconditions, actor roles, actions, input constraints, and expected outcomes from unstructured scenario text, forming the semantic foundation for test scenario generation.

Constraint-Based Scenario Synthesis

- The extracted semantic elements are passed through a constraint-based synthesis module that automatically applies Boundary Value Analysis (BVA) and Equivalence Partitioning (EP) techniques. This module generates a comprehensive set of positive, negative, and boundary test scenarios by systematically exploring valid, invalid, and edge input combinations derived from the identified constraints.

Abstract Test Model Construction

- After scenario synthesis and edge case prediction, an Abstract Test Model (ATM) is constructed as a framework-agnostic intermediate representation of all generated test scenarios. The ATM decouples test logic from any specific automation framework, storing test steps, assertions, input data, and expected outcomes in a structured, reusable format that can be independently translated into multiple target frameworks.

Multi-Framework Code Generation

- The Abstract Test Model is passed through dedicated code generation engines for each target framework — Selenium, Cypress, and Playwright. Each engine translates the ATM into idiomatic, type-safe, executable test scripts following the best practices and conventions of the respective framework. Static analysis is applied during generation to verify syntactic correctness and type safety before the scripts are committed to the test repository.

Mutation Testing Validation & Gap Detection

- Generated test suites are evaluated using mutation testing techniques that introduce small, controlled faults into the source code and measure whether the generated tests detect them. The mutation score provides a quantitative measure of test suite fault detection capability. Tests that fail to detect mutations are flagged as coverage gaps, and the system automatically triggers regeneration of additional scenarios to close identified gaps, feeding back into the synthesis module.

3.3 Tools and Platforms with Justification

Several technologies will be used to implement the proposed intelligent test framework.

- Python will be used as the primary programming language because it provides extensive libraries for NLP, machine learning, and code generation development.
- spaCy / Stanford NLP will be used for semantic parsing of Gherkin acceptance criteria using NER and dependency parsing.
- Scikit-learn will be used to implement the Random Forest classifier for ML-based edge case prediction.
- FastAPI will be used to build the backend REST API that connects the React dashboard with the NLP and ML processing modules.
- React will be used to develop the human-in-the-loop review dashboard for QA engineers.
- AST (Abstract Syntax Tree) libraries will be used to construct and manipulate the Abstract Test Model and generate syntactically correct test code.
- Selenium, Cypress, and Playwright serve as the target code generation frameworks for which type-safe test scripts are produced.
- MutPy / Cosmic Ray will be used for mutation testing to evaluate generated test suite effectiveness and detect coverage gaps.
- MySQL / PostgreSQL will be used to store historical defect data, generate test scenarios, and mutation testing results.
- Jenkins / GitHub Actions will be used to integrate generated tests into CI/CD pipelines for automated execution.

Tool / Platform	Purpose	Alternative Tools	Justification for Selection
Python	Core development language for implementing machine learning models and data processing.	Java, R	Strongest ecosystem for NLP and ML libraries; faster prototyping for research

spaCy / Stanford NLP	Semantic parsing, NER, and dependency parsing of Gherkin steps	NLTK, TextBlob	spaCy provides industrial-strength NLP with pre-trained models; faster and more accurate than NLTK for entity extraction
Scikit-learn	Random Forest classifier for ML-based edge case prediction	TensorFlow, PyTorch	More suitable for classical ML classification tasks; simpler to train and interpret than deep learning frameworks
Pandas & NumPy	Feature extraction and preprocessing of defect datasets	Apache Spark, MATLAB	Efficient data manipulation for medium-scale datasets; seamless integration with Python ML libraries
FastAPI	Backend REST API connecting dashboard with NLP/ML modules	Django, Flask	FastAPI offers high performance, async support, and automatic API documentation suitable for real-time processing
React	Human-in-the-loop review dashboard for QA engineers	Angular, Vue.js	React provides flexible component-based UI development; widely adopted in industry for interactive dashboards
AST Libraries (Python ast / Babel)	Abstract Test Model construction and type-safe code generation	Manual string templating	AST manipulation ensures syntactically correct, analyzable, and maintainable code generation across frameworks
Selenium / Cypress / Playwright	Target frameworks for generated test script output	Appium, WebdriverIO	These three frameworks cover the majority of web automation industry usage; ensuring broad applicability of generated tests

MutPy / Cosmic Ray	Mutation testing to validate generated test suite quality	PITest, Stryker	Provides reliable mutation scores to measure fault detection capability; integrates well with Python-based pipelines
MySQL / PostgreSQL	Storage of defect history, test scenarios, and evaluation results	MongoDB, SQLite	Relational structure suits structured defect and test data; supports efficient analytical queries
Jenkins / GitHub Actions	CI/CD pipeline integration and automated test execution	GitLab CI, CircleCI	Widely adopted in industry; flexible pipeline configuration and strong plugin ecosystem
Git / GitHub	Version control and collaboration	SVN, Bitbucket	Most widely adopted distributed version control; supports research collaboration and test artefact management
Docker	Containerization of experimental environment	Virtual Machines, Kubernetes	Lightweight containerization ensures reproducibility and consistent environment across development and testing phases

Table 1: Tools and Platforms Used for System Implementation

These tools were selected because they are widely used in modern DevOps environments and support integration with automated testing pipelines.

3.4 Data Collection and Ethical Considerations

The dataset used in this research will be collected from real Agile software projects, open-source repositories, and simulated development environments. Since the proposed component focuses on intelligent test case generation from Gherkin acceptance criteria, the data sources will primarily consist of Gherkin feature files, historical software defect records, and manually designed test suites used as ground truth for evaluation.

Data Sources

- The data will be collected from the following sources:

1. Open-Source Agile Project Repositories
   Publicly available GitHub repositories containing Gherkin. feature files from BDD-based projects such as e-commerce, banking, booking, and authentication systems will be collected. These provide realistic acceptance criteria for training and testing the semantic parsing engine.
2. Open-Source Bug Tracking Datasets
   Publicly available defect datasets from sources such as the Mozilla Bug Database, Apache JIRA, and GitHub Issues will be used to train the Random Forest edge case prediction model. These datasets provide historical defect patterns linked to specific functional areas and input conditions.
3. Manually Created Gherkin Scenarios
   Additional Gherkin acceptance criteria will be manually authored to cover diverse constraint types, boundary conditions, and edge case patterns not sufficiently represented in open-source repositories. These will supplement training data for the constraint synthesis and ML modules.
4. Expert-Designed Test Suites
5. Manually designed test suites from experienced QA engineers will be collected as ground truth for evaluating the quality and completeness of AI-generated test scenarios and code.

Types of Data Collected

The following data types will be collected for each automated test execution:

- Gherkin .feature file content (Given-When-Then steps)
- Extracted semantic elements (preconditions, actions, constraints, expected outcomes)
- Boundary value and equivalence class parameters
- Historical defect records (feature area, defect type, input conditions, severity)
- Generated test scenarios (positive, negative, edge case)
- Generated test code for Selenium, Cypress, and Playwright
- Mutation testing scores and coverage gap reports
- Expert quality ratings from evaluation surveys

Expected Dataset Size

The study aims to collect and process approximately 500 to 1,000 Gherkin acceptance criteria scenarios during the experimental phase, sourced across multiple application domains. Among these:

- Approximately 300 to 600 defect records will be used to train and validate the Random Forest edge case prediction model
- A balanced labeled dataset of 200 to 400 expert-reviewed test scenario pairs (AI-generated vs. manually designed) will be prepared for comparative quality evaluation

Data Labeling

- To train the ML-based edge case prediction model, historical defect records will be reviewed and labeled according to defect type, functional area, input boundary conditions, and risk level. Each defect instance will be assigned a structured feature vector used for Random Forest classifier training. Generated test scenarios will additionally be reviewed and rated by Senior QA engineers on dimensions of correctness, completeness, and framework-specific code quality.

Ethical Considerations

- This research primarily uses technical system-generated data and publicly available open-source project artefacts rather than personal user information. Therefore, the risk of privacy violations is minimal. However, any sensitive project-specific identifiers or confidential business logic appearing in Gherkin files or defect records collected from industry partners will be fully anonymized before analysis.

- The collected data will be used strictly for academic research purposes and stored securely in role-restricted repositories. No personal user data, credentials, or private business information will be intentionally collected as part of this research. All AI-generated test artefacts will be clearly labeled to ensure human accountability and responsible use throughout the research process.

3.5 Validation Strategy and Performance Metrics

- The performance of the proposed system will be evaluated through experimental testing using Gherkin acceptance criteria datasets, expert review sessions, and comparative analysis against manually designed test suites.
- The semantic parsing engine will be evaluated using standard NLP metrics including Precision, Recall, and F1-score, measuring the accuracy of extracted preconditions, actions, constraints, and expected outcomes against manually annotated ground truth Gherkin scenarios.
- The ML-based edge case prediction model will be evaluated using classification metrics including Accuracy, Precision, Recall, and F1-score, assessing the model's ability to

correctly predict high-risk edge cases and negative scenarios from historical defect feature vectors.

- The constraint-based scenario synthesis module will be evaluated by measuring scenario coverage completeness, calculating the percentage of boundary values, equivalence classes, and edge cases correctly generated compared to expert-designed test scenario sets.
- The multi-framework code generation engine will be evaluated based on code correctness rate, type-safety compliance, and executability rate, measuring the percentage of generated Selenium, Cypress, and Playwright scripts that execute successfully without syntax or runtime errors.
- The mutation testing validation module will be evaluated using mutation score improvement, measuring the increase in fault detection capability of AI-generated test suites compared to manually designed suites across multiple application domains.
- The overall system performance will be assessed through the following key metrics:

• **Metric**	• **Target**
• Semantic parsing F1-score	• ≥ 85%
• Edge case prediction accuracy	• ≥ 80%
• Code generation executability rate	• ≥ 90%
• Mutation score improvement vs. manual	• ≥ 20–30% uplift
• Scenario generation latency	• < 10 seconds per Gherkin scenario
• Expert quality rating (Likert scale)	• ≥ 4.0 / 5.0

- Additionally, a comparative A/B evaluation will be conducted on real Agile sprint cycles, measuring reduction in manual test design effort, improvement in branch coverage, and decrease in escaped defects when using the proposed system versus traditional manual test case design approaches.

4. HIGH-LEVEL SYSTEM ARCHITECTURE

4.1 Overview of the Proposed Solution and Its Components

The proposed system, NEXTGEN QA, is an intelligent automated testing framework designed to improve the reliability, coverage, and efficiency of Agile software development and CI/CD pipelines. The system integrates multiple intelligent modules that support requirement extraction, automated test generation, failure analysis, and quality prediction.

The architecture of the proposed solution consists of four main components.

The first component is Voice/NLP Intelligence, which processes voice discussions and Agile meeting transcripts to extract structured software requirements using speech-to-text and natural language processing techniques, producing INVEST-validated user stories and Gherkin acceptance criteria.

The second component is Intelligent Test Case Generation with Multi-Framework Code Synthesis, which automatically transforms Gherkin acceptance criteria into comprehensive test scenarios and generates type-safe, idiomatic test code for multiple automation frameworks. This component applies semantic parsing using NER and dependency parsing to extract test elements, applies constraint-based boundary value analysis and equivalence partitioning for scenario synthesis, uses a Random Forest ML classifier trained on historical defect data to predict high-risk edge cases, constructs a framework-agnostic Abstract Test Model, and generates executable test scripts for Selenium, Cypress, and Playwright simultaneously. This component represents the main focus of this research.

The third component is Failure Analysis and Self-Healing Intelligence, which focuses on analyzing test execution failures within CI/CD pipelines. This component identifies root causes of test failures, generates automated repair strategies, predicts unstable flaky tests, and continuously improves test stability through adaptive learning mechanisms.

The fourth component is Quality Prediction and RTM Generation, which analyzes testing results and automatically generates a Requirements Traceability Matrix (RTM) and software quality insights for project stakeholders, linking requirements to test coverage and defect outcomes.

Together, these four components create an intelligent end-to-end testing ecosystem that reduces manual QA effort, improves test coverage and code quality, and increases the overall reliability of automated testing in modern Agile CI/CD environments.

4.2 System Diagram of Components

The high-level system architecture consists of four interconnected components that exchange data throughout the testing lifecycle.

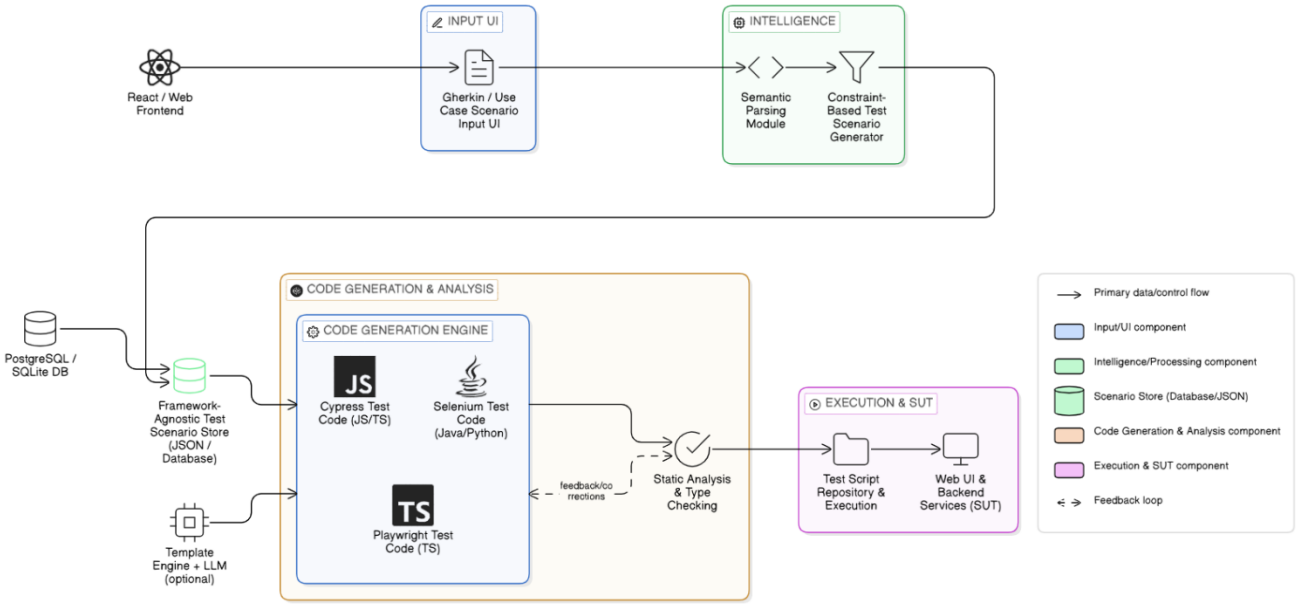


Figure 1: High-Level Architecture of the NEXTGEN QA System

4.3 Individual Component and Its Parts

The Intelligent Test Case Generation with Multi-Framework Code Synthesis component consists of four interconnected subsystems as illustrated in the architecture diagram above. Each subsystem is responsible for a distinct stage of the test generation pipeline.

INPUT UI Subsystem

The Input UI subsystem provides the interface through which QA engineers interact with the system. A React/Web Frontend connects to a Gherkin/Use Case Scenario Input UI, allowing engineers to upload or manually enter Gherkin .feature files and use case scenarios. This subsystem handles input validation and preprocessing before passing structured scenario text to the Intelligence subsystem.

INTELLIGENCE Subsystem

The Intelligence subsystem is responsible for understanding and transforming raw Gherkin acceptance criteria into structured test scenarios. It consists of two sequential modules:

- Semantic Parsing Module - applies NER and dependency parsing using spaCy/Stanford NLP to extract preconditions, actor roles, actions, constraints, and expected outcomes from each Gherkin step.

- Constraint-Based Test Scenario Generator - applies Boundary Value Analysis (BVA) and Equivalence Partitioning (EP) to synthesize comprehensive positive, negative, and edge case test scenarios from the extracted semantic elements. A Random Forest ML classifier further enriches scenarios with high-risk edge case predictions derived from historical defect data.

The output of this subsystem is a set of abstract test scenarios stored in the Framework-Agnostic Test Scenario Store (JSON / Database) backed by PostgreSQL / SQLiteDB,which acts as the central data exchange point between the Intelligence and Code Generation subsystems.

CODE GENERATION & ANALYSIS Subsystem

The Code Generation & Analysis subsystem transforms abstract test scenarios into executable, type-safe test scripts for multiple automation frameworks. It contains a nested Code Generation Engine with three parallel generators:

- Cypress Test Code (JS/TS) - generates idiomatic JavaScript/TypeScript test scripts for the Cypress framework
- Selenium Test Code (Java/Python) - generates type-safe Java or Python test scripts for the Selenium WebDriver framework
- Playwright Test Code (TS) - generates TypeScript test scripts for the Playwright automation framework

An optional Template Engine + LLM module supplements generation for complex scenarios. All generated scripts pass through a Static Analysis & Type Checking module that validates syntactic correctness, type safety, and executability. A dashed feedback/corrections loop returns type errors and static analysis findings back to the Code Generation Engine for automatic repair before final output.

EXECUTION & SUT Subsystem

The Execution & SUT subsystem handles the final deployment and execution of generated test scripts. The Test Script Repository & Execution module stores approved scripts and triggers their execution within CI/CD pipelines. Tests are executed against the Web UI &

Backend Services (SUT — System Under Test), validating both frontend and backend functionality. Execution results feed back into the system for coverage analysis and quality evaluation.

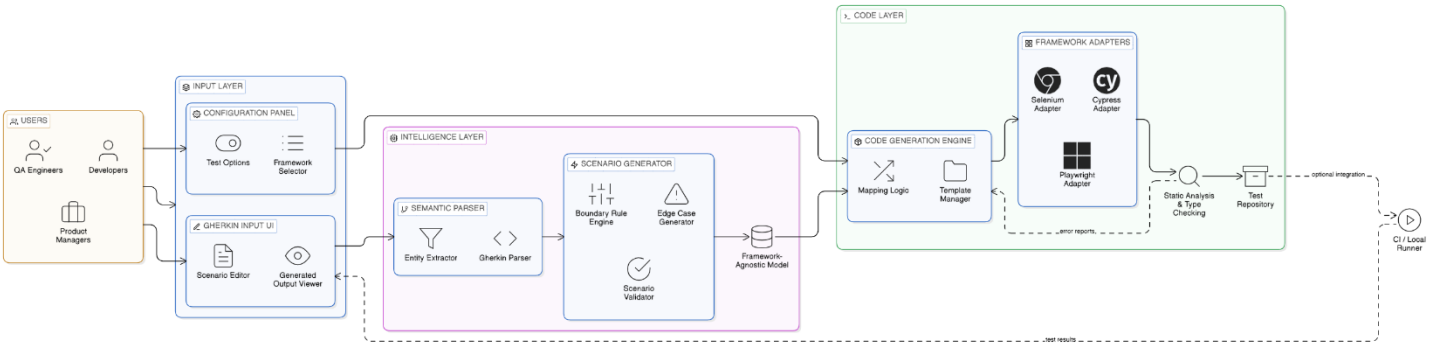


Figure 2: System Architecture Diagram for Individual Component

The Intelligent Test Case Generation with Multi-Framework Code Synthesis component consists of several submodules that transform Gherkin acceptance criteria into comprehensive, executable test scripts across multiple automation frameworks.

These modules include:

- Gherkin Input & Preprocessing Module - collects and validates Gherkin. feature files uploaded through the React Dashboard, tokenizes scenario text, and structures each Given-When-Then step for semantic processing.
- Semantic Parsing Module - applies NER and dependency parsing using spaCy/Stanford NLP to extract preconditions, actor roles, actions, input constraints, and expected outcomes from each Gherkin step.
- Constraint-Based Scenario Synthesis Module - applies Boundary Value Analysis (BVA) and Equivalence Partitioning (EP) to automatically generate comprehensive positive, negative, and boundary test scenarios from extracted semantic elements.
- ML Edge Case Prediction Module - applies a Random Forest classifier trained on historical defect data to predict high-risk edge cases and negative scenarios likely to expose software faults, enriching the synthesized scenario set.
- Abstract Test Model Construction Module - builds a framework-agnostic intermediate representation of all generated test scenarios, decoupling test logic from any specific automation framework.

- Multi-Framework Code Generation Engine - translates the Abstract Test Model into idiomatic, type-safe executable test scripts in parallel for Selenium, Cypress, and Playwright using AST-based code generation.
- Static Analysis & Type Checking Module - validates generated scripts for syntactic correctness, type safety, and executability before committing to the test repository, with a feedback loop back to the generation engine for corrections.
- Mutation Testing & Validation Module - evaluates generated test suite fault detection capability using mutation scores, identifies coverage gaps, and triggers automatic scenario regeneration to improve overall coverage.

4.4 Component Interactions

Each component of the NEXTGEN QA system interacts with others to create a completely intelligent automated testing ecosystem.

Component 1 (Voice/NLP Intelligence) extracts structured software requirements from Agile voice discussions and meeting transcripts and passes validated Gherkin acceptance criteria to Component 2.

Component 2 (Intelligent Test Case Generation) receives Gherkin acceptance criteria, semantically parses them, synthesizes comprehensive test scenarios, predicts edge cases using ML, and generates type-safe test scripts for Selenium, Cypress, and Playwright. The generated and validated test scripts are passed to Component 3 for execution within CI/CD pipelines.

Component 3 (Failure Analysis & Self-Healing) receives test execution results, logs, and failure artifacts from CI/CD pipelines. It identifies root causes of failures, applies self-healing repairs to unstable test scripts, and returns improved, stabilized test scripts and failure pattern insights back to Component 2 to continuously improve generated test quality.

Component 4 (Quality Prediction & RTM Generation) receives test execution outcomes, mutation scores, and coverage reports from Component 2 and Component 3. It generates software quality predictions, requirement traceability matrices, and coverage insights for project stakeholders, and feeds coverage gap reports back to Component 2 to trigger additional test scenario generation.

4.5 Data Flow Explanation

The system follows a sequential data flow starting from Gherkin acceptance criteria input through to validated, CI/CD-ready test scripts.

First, QA engineers upload Gherkin feature files through the React Dashboard. The Semantic Parsing Module processes each Given-When-Then step to extract structured semantic elements including preconditions, actions, constraints, and expected outcomes.

These structured elements are passed to the Constraint-Based Scenario Synthesis Module, which applies boundary value analysis and equivalence partitioning to generate a comprehensive set of test scenarios. Simultaneously, the ML Edge Case Prediction Module enriches this scenario set with high-risk edge cases predicted from historical defect patterns.

All synthesized scenarios are consolidated into a framework-agnostic Abstract Test Model stored in the PostgreSQL/SQLite Test Scenario Store. The Multi-Framework Code Generation Engine reads from this store and produces idiomatic, type-safe test scripts for Selenium, Cypress, and Playwright in parallel.

Generated scripts pass through Static Analysis and Type Checking, with corrections fed back to the generation engine. Approved scripts are stored in the Shared Test Repository and executed within CI/CD pipelines. Mutation testing results and coverage gap reports are fed back into the synthesis module to regenerate missing scenarios, creating a continuously improving test generation cycle.

4.6 Scalability and Security Considerations

The proposed system is designed to support large-scale Agile development environments where hundreds of Gherkin acceptance criteria scenarios may be processed and test scripts generated continuously across multiple sprints.

To ensure scalability, the architecture adopts a modular, API-driven design built on FastAPI, allowing each submodule — semantic parsing, ML prediction, code generation, and mutation testing — to be independently scaled based on workload demand. The framework-agnostic Abstract Test Model enables seamless extension to additional automation frameworks beyond Selenium, Cypress, and Playwright without redesigning the core pipeline. The Random Forest classifier and scenario synthesis modules can be periodically retrained using updated defect datasets to maintain prediction accuracy as projects evolve.

From a security perspective, the system processes only project Gherkin files and anonymized historical defect records, ensuring no personal user data or production system credentials are handled. All generated test artefacts, defect datasets, and ML model training data are stored in role-restricted, access-controlled repositories. Any sensitive project-specific identifiers or confidential business logic appearing in Gherkin files collected from industry partners are fully anonymized before processing. All AI-generated test scripts are clearly labeled and subject to human QA engineer review and approval before deployment into live CI/CD pipelines, ensuring responsible and auditable use of the system throughout the research process.

5. USER REQUIREMENTS

5.1 Stakeholders

The proposed intelligent automated testing framework will involve several stakeholders who interact with different parts of the system.

Stakeholder	Description
QA Engineers / Testers	Primary users of the system. Upload Gherkin feature files, review AI-generated test scenarios, approve or reject generated test scripts through the human-in-the-loop React Dashboard, and manage the final test suite quality.
Software Developers	Provide Gherkin acceptance criteria as part of the Agile development process. Receive generated test scripts for review and use them during development to validate feature implementations early.
DevOps Engineers	Integrate the generated test scripts into CI/CD pipelines using Jenkins or GitHub Actions. Manage the Shared JSON Test Repository and ensure generated tests are executed correctly within the automated deployment workflow.
Project Managers	Monitor test coverage metrics, mutation scores, and scenario generation progress through quality dashboards. Use coverage reports and RTM outputs to track requirement-to-test traceability and overall testing completeness.
Research System	The intelligent pipeline module that automatically performs semantic parsing, constraint-based scenario synthesis, ML-based edge case prediction, Abstract Test Model construction, multi-framework code

(Automated Agent)	generation, and mutation testing validation without human intervention.

Table 2: Stakeholder Analysis

5.2 Functional Requirements

The system must support the following functional capabilities.

FR1: The system shall accept Gherkin feature files uploaded by QA engineers through the React Dashboard and validate the syntactic correctness of Given-When-Then scenario structure before processing.

FR2: The system shall apply Named Entity Recognition (NER) and dependency parsing to each Gherkin step to extract structured semantic elements including preconditions, actor roles, actions, input constraints, and expected outcomes.

FR3: The system shall automatically apply Boundary Value Analysis (BVA) and Equivalence Partitioning (EP) techniques to generate comprehensive positive, negative, and boundary test scenarios from the extracted semantic elements.

FR4: The system shall apply a trained Random Forest classifier to predict high-risk edge cases and negative test scenarios based on historical defect data, enriching the synthesized scenario set with ML-driven risk predictions.

FR5: The system shall construct a framework-agnostic Abstract Test Model (ATM) that represents all generated test scenarios in a unified, reusable intermediate format decoupled from any specific automation framework.

FR6: The system shall generate idiomatic, type-safe, executable test scripts simultaneously for Selenium, Cypress, and Playwright frameworks from the Abstract Test Model using AST-based code generation engines.

FR7: The system shall perform static analysis and type checking on all generated test scripts to verify syntactic correctness and executability before storing them in the Shared Test Repository.

FR8: The system shall evaluate the fault detection capability of generated test suites using mutation testing and calculate mutation scores to measure test suite effectiveness.

FR9: The system shall automatically detect coverage gaps in generated test suites based on mutation testing results and trigger regeneration of additional test scenarios to fill identified gaps.

FR10: The system shall store all approved, validated test scenarios and generated test scripts in the Shared JSON Test Repository for consumption by CI/CD pipelines and downstream NEXTGEN QA components including the Quality Prediction and RTM Generation module.

5.3 Non Functional Requirements

Non-functional requirements define system quality attributes such as performance, reliability, and security.

Performance:

The system shall complete semantic parsing, scenario synthesis, ML edge case prediction, and test code generation within 10 seconds per Gherkin scenario to ensure it integrates smoothly into fast-paced Agile sprint cycles without introducing delays to CI/CD pipeline execution.

Scalability:

The system shall support large-scale Agile development environments where hundreds of Gherkin acceptance criteria scenarios may be processed per sprint. The modular FastAPI-based architecture shall allow individual submodules semantic parsing, ML prediction, and code generation engines to be independently scaled based on workload demand without affecting overall system performance.

Reliability:

The system shall maintain a code generation executability rate of ≥ 90%, ensuring that generated Selenium, Cypress, and Playwright test scripts execute successfully without syntax or runtime errors. The ML edge case prediction model shall maintain a classification accuracy of ≥ 80% to minimize missed high-risk scenarios and incorrect test scenario generation.

Security:

All Gherkin input files, historical defect datasets, generated test scripts, and mutation testing results shall be stored in role-restricted, access-controlled repositories. Any sensitive project-specific identifiers or confidential business logic appearing in Gherkin files shall be fully anonymized before processing. No personal user data or production system credentials shall be collected or stored by the system.

Usability:

The React Dashboard shall provide QA engineers with a clear, intuitive human-in-the-loop interface for reviewing, approving, and rejecting AI-generated test scenarios and scripts. Generated test artefacts shall be clearly labeled as AI-generated to maintain human accountability throughout the test design process.

Maintainability:

The system architecture shall adopt a modular, loosely coupled design that allows new automation frameworks to be added as additional code generation engines without modifying existing modules. The Random Forest classifier shall support periodic retraining using updated defect datasets to maintain prediction accuracy as project defect patterns evolve over time.

Portability:

The system shall be containerized using Docker to ensure consistent execution across different development, testing, and production environments, guaranteeing reproducibility of research experiments and ease of deployment in diverse CI/CD infrastructure setups.

6. COMMERCIALIZATION PLAN

6.1. Target Market and Customer Persona

The proposed system targets software development companies that rely heavily on automated testing and CI/CD pipelines. These organizations typically follow Agile and DevOps development practices and require reliable automated testing frameworks to maintain

software quality while delivering frequent software updates:

- Potential customers include:
- Software development companies
- Quality assurance teams
- DevOps teams
- Technology startups
- Enterprises maintaining large automated test suites
-

A typical customer persona for the proposed solution is a QA Automation Engineer or DevOps Engineer responsible for maintaining automated test pipelines. These professionals face challenges such as flaky tests, frequent test failures, and long debugging times. The proposed system helps these users reduce debugging effort and improve the reliability of automated testing.

6.2. Value Proposition

- The proposed system provides significant value to Agile software development teams by improving the efficiency, coverage, and consistency of automated test design processes.

- The key value propositions include:

    - Automatic transformation of Gherkin acceptance criteria into comprehensive, ready-to-run test scripts eliminating hours of manual test writing per sprint
    - Intelligent generation of boundary value, equivalence class, and high-risk edge case scenarios that are frequently missed in manual test design
    - Multi-framework code generation produces idiomatic Selenium, Cypress, and Playwright scripts simultaneously from a single input reducing framework-specific rework
    - ML-driven risk prediction that prioritizes high defect probability test scenarios, improving fault detection rates early in the development cycle
    - Continuous mutation testing validation that automatically identifies and fills coverage gaps without manual intervention
    - Significant reduction in manual QA effort, test maintenance costs, and time to test across Agile sprint cycles
    - By automating the full pipeline from acceptance criteria to executable test code, the system helps organizations save time, reduce test design costs, improve test coverage quality, and accelerate CI/CD pipeline reliability.

6.3. Revenue Model

The proposed system can be commercialized as a cloud-based Software-as-a-Service (SaaS) platform or as a CI/CD plugin integrated with existing Agile DevOps toolchains.

- Possible revenue models include:

    - Subscription-based licensing for Agile development teams based on the number of Gherkin scenarios processed per month
    - Enterprise licensing for large organizations requiring multi-project, multi-team test generation pipelines
    - Plugin-based integration services for CI/CD tools such as Jenkins, GitHub Actions, and GitLab CI
    - Framework extension services for organizations requiring support for additional automation frameworks beyond Selenium, Cypress, and Playwright
    - Organizations may subscribe to the platform based on the number of Gherkin

scenarios processed, the number of automation frameworks targeted, or the size of the QA engineering team.

6.4. Cost Estimation

The initial development and deployment cost of the proposed Intelligent Test Case Generation component is estimated to be approximately Rs. 32,000 for building a functional research prototype. The estimated budget allocation is as follows:

Cost Category	**Estimated Cost**
Cloud hosting for backend API and database (PostgreSQL/MySQL - 6 months)	Rs. 12,500
LLM inference credits (CodeLlama/Hugging Face -API usage)	Rs. 18,500
Development environment (VS Code Pro, PyCharm Community, premium plugins)	Rs. 10,000
Node.js/Python server hosting (VPS for testing)	Rs. 11,500
Static analysis tools (TypeScript/ESLint Pro licenses if needed)	Rs. 5,000
Internet & utilities for development (shared cost for component)	Rs. 8,000
Test automation frameworks setup (Selenium Grid, Playwright browsers)	Rs. 8,500
Miscellaneous (unexpected costs, small equipment, adapters)	Rs. 6,000
Total Budget	**Rs. 80,000**

Table 3: Cost Estimation for System Development

Total Estimated Cost: **Rs. 80,000**

6.5. Pricing Strategy

The system will follow a subscription-based pricing strategy to generate sustainable revenue while remaining accessible and affordable for small, medium, and large software development teams operating within Agile environments.

The pricing tiers are structured to reflect the scale of Gherkin scenario processing, the number of supported automation frameworks, and the level of CI/CD integration required by each organization.

Example pricing structure:

Plan	**Monthly Price**	**Target Users**
Basic Plan	$15–$25 / month	Small Agile teams with limited test suites and single-framework needs
Professional Plan	$50–$80 / month	Medium-sized DevOps teams requiring multi-framework generation and ML edge case prediction
Enterprise Plan	Custom pricing	Large organizations requiring advanced CI/CD integration, dedicated support, and multi-project pipelines

Table 4: Pricing Strategy and Subscription Plans

- Key considerations of this pricing model include:
  - Basic Plan provides access to core Gherkin-to-test-script generation with a single automation framework (Selenium, Cypress, or Playwright)
  - Professional Plan unlocks multi-framework code generation, Random Forest-based edge case prediction, and mutation testing coverage analysis
  - Enterprise Plan includes unlimited scenario processing, on-premises deployment options, custom ML model retraining, and dedicated SLA-backed technical support
  - At an average of 50 Professional Plan subscribers, the platform can recover the Rs. 32,000 prototype investment within the first operational quarter

  - With this tiered pricing model, the system can sustainably recover initial development costs while continuously funding platform improvements, ML model retraining, and framework extension services over the long term.

.

6.6. Competitive Advantage

Although existing automated testing tools such as Selenium, Cypress, and Playwright provide robust test execution capabilities, most of these tools do not offer intelligent test design automation, ML-driven edge case prediction, or multi-framework code generation from natural language inputs.

The proposed system offers several key competitive advantages over traditional test automation tools:

- **NLP-Driven Test Generation** - Automatically transforms Gherkin acceptance criteria into executable test scripts, eliminating the need for manual test script authoring and reducing human error in test design
- **ML-Based Edge Case Prediction** - Employs a Random Forest classifier trained on historical defect datasets to intelligently predict and generate high-risk boundary and edge case scenarios that manual testers frequently overlook
- **Multi-Framework Code Generation** - Simultaneously produces idiomatic, ready-to-run test scripts for Selenium, Cypress, and Playwright from a single Gherkin input — a capability no current open-source tool offers natively
- **Mutation Testing Coverage Validation** - Continuously evaluates generated test suite strength using mutation testing, automatically identifying and filling coverage gaps without manual intervention
- **Seamless CI/CD Integration** - Designed to integrate directly into Jenkins, GitHub Actions, and GitLab CI pipelines, enabling fully automated test generation as part of the Agile sprint delivery cycle
- **Human-in-the-Loop Oversight** - The React Dashboard provides QA engineers with transparent control to review, approve, or reject AI-generated scenarios, maintaining human accountability in the testing process

These capabilities collectively position the system as a next-generation Agile QA automation platform that goes significantly beyond the test execution focus of traditional tools, delivering intelligent test design, coverage assurance, and continuous quality improvement throughout the software development lifecycle.

6.7. Intellectual Property

- The proposed framework may involve intellectual property related to NLP-based semantic parsing algorithms and machine learning models for edge case prediction trained on historical defect data.
- The research outputs may be protected through software copyrights or potential patent applications for the novel unified pipeline combining constraint-based scenario synthesis with framework-agnostic, type-safe test code generation for multiple automation frameworks simultaneously.

- The system may also be released under a controlled licensing model to protect the proprietary semantic parsing engine and Random Forest prediction module, while allowing commercial deployment through integration with CI/CD pipelines and third-party QA platforms.

7. BUDGET AND JUSTIFICATION

The estimated budget required to design, develop, and evaluate the proposed Intelligent Test Case Generation research prototype is approximately Rs. 80,000.

7.1 Hardware Costs

Item	Estimated Cost
Development workstation for NLP pipeline and ML model training	Rs. 15,000
Additional testing devices for cross-browser Selenium/Playwright execution	Rs. 5,000

Table 5: Budget Allocation for Hardware

Total Hardware Cost: Rs. 20,000

Justification:
Hardware resources are required to run Random Forest model training experiments, execute multi-framework test generation pipelines, and process Gherkin datasets locally during early-stage prototype development.

7.2 Software Licenses

Item	Estimated Cost
Development tools and IDE licenses (VS Code Pro, PyCharm plugins)	Rs. 3,000
Testing framework plugins and integrations (ESLint Pro, Selenium Grid setup)	Rs. 2,000
Static analysis and code quality tools	Rs. 2,000

Table 6: Budget Allocation for Software

Total Software Cost: Rs. 7,000

Justification:
Although the majority of technologies used including spaCy, FastAPI, Selenium, Cypress,

and Playwright are open-source, select development tools and CI/CD integration plugins require paid licensing for professional-grade functionality.

7.3 Cloud Services

Item	Estimated Cost
Cloud hosting for FastAPI backend and PostgreSQL database (6 months)	Rs. 10,000
LLM inference credits (CodeLlama / Hugging Face API usage)	Rs. 12,000
VPS hosting for Node.js / Python testing server	Rs. 8,000
Database hosting and experiment environment storage	Rs. 5,000

Table 7: Budget Allocation for Cloud Services

Total Cloud Cost: Rs. 35,000

Justification:

Cloud services are essential to host the test generation API, store Gherkin input datasets and generated test artefacts, execute LLM-based code generation, and simulate CI/CD pipeline integration experiments throughout the research evaluation phase.

7.4 Human Effort Cost

Role	Estimated Cost
System integration, testing support and QA validation	Rs. 8,000
Internet & utilities for development (shared component cost)	Rs. 5,000
Miscellaneous (unexpected costs, small equipment, adapters)	Rs. 5,000

Table 8: Budget Allocation for Human Effort

Total Operational Cost: Rs. 18,000

Justification:

Core system design, NLP semantic parsing, Random Forest model training, and multi-

framework code generation are implemented directly by the researcher. Operational costs cover integration support, utilities, shared infrastructure, and unforeseen miscellaneous expenses during the development period.

7.5 Total Estimated Budget

Category	Cost
Hardware	Rs. 20,000
Software	Rs. 7,000
Cloud Services	Rs. 35,000
Human Effort	Rs. 18,000

Table 9: Total Estimation Budget

Total Estimated Budget: Rs. 80,000

8. WORK BREAKDOWN STRUCTURE (WBS)

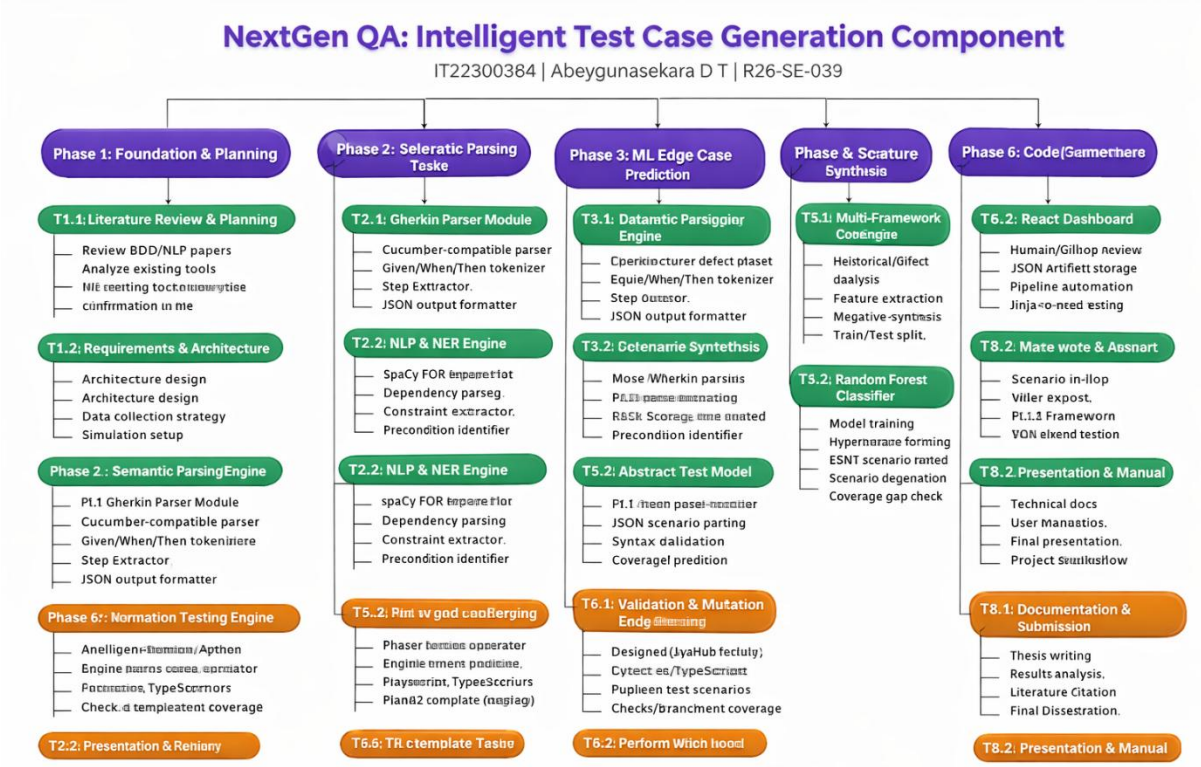


Figure 3: Work Breakdown Structure Diagram

9. GANTT CHART

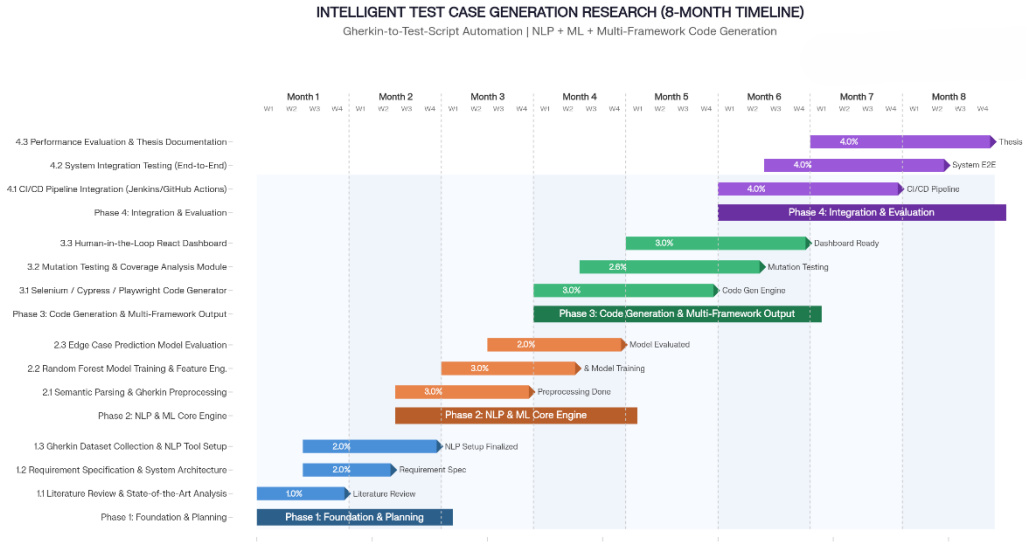


Figure 4: Gantt Chart

10. REFERENCES LIST

References

[1] S. Kamalakar et al., *Automatically Generating Tests from Natural Language Descriptions of Software Behavior*, Virginia Tech, Blacksburg, VA, 2013. Available: https://vtechworks.lib.vt.edu/handle/10919/23907

[2] A. Kugler, S. Kowalewski, and B. Rumpe, *Test Case Generation from Natural Language Requirements for Embedded Systems with Semantic Role Labeling*, RWTH Aachen University, Aachen, Germany, 2025. Available: https://publications.rwth-aachen.de/record/1010169

[3] V. Garousi et al., *Agentic AI for Behaviour-Driven Development Testing Using Large Language Models*, SCITEPRESS, 2025. Available: https://www.scitepress.org/Papers/2025/133744/133744.pdf

[4] G. Fraser and A. Arcuri, "Automated Test Case Generation Using Search-Based Techniques," *IEEE Transactions on Software Engineering*, IEEE Computer Society, 2015. Available: https://doi.org/10.1109/TSE.2013.75

[5] testRigor Inc., "AI-Driven Edge Case Prediction and Test Case Generation," testRigor Research Blog, 2025. Available: https://testrigor.com/blog/what-are-edge-test-cases/

[6] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed., John Wiley & Sons, 2011.

[7] M. Leotta et al., "Comparing the Maintainability of Selenium WebDriver Test Suites Employing Different Locator Strategies," *Empirical Software Engineering*, Springer, 2016. Available: https://doi.org/10.1007/s10664-015-9409-3

[8] Y. Jia and M. Harman, "An Analysis and Survey of the Development of Mutation Testing," *IEEE Transactions on Software Engineering*, IEEE Computer Society, 2011. Available: https://doi.org/10.1109/TSE.2010.62

11. APPENDICES

The appendices provide additional supporting materials related to the proposed Intelligent Test Case Generation system. These materials include research instruments used during the study, extended system diagrams, preliminary interface designs, risk evaluation, and ethical considerations.

Appendix A – Survey Instruments

A short survey will be conducted with software testers, QA engineers, and Agile development team members to understand common challenges in manual test case design and acceptance criteria coverage.

Example survey questions include:

1. How frequently do your QA teams manually write test scripts from Gherkin acceptance criteria in each sprint?
2. What are the most common challenges your team faces when converting user stories into automated test cases?
3. How much time does your team spend writing and maintaining test scripts per sprint cycle?
4. Do you currently use any tools that automatically generate test scripts from Agile user stories or Gherkin scenarios?
5. How valuable would an automated system that generates Selenium, Cypress, and Playwright test scripts directly from Gherkin acceptance criteria be for your development process?
6. How confident are you in the edge case and boundary value coverage of your current manually written test suites?

Appendix B – Detailed System Diagrams

This appendix contains detailed diagrams describing the proposed Intelligent Test Case Generation framework, including High-level system architecture diagram

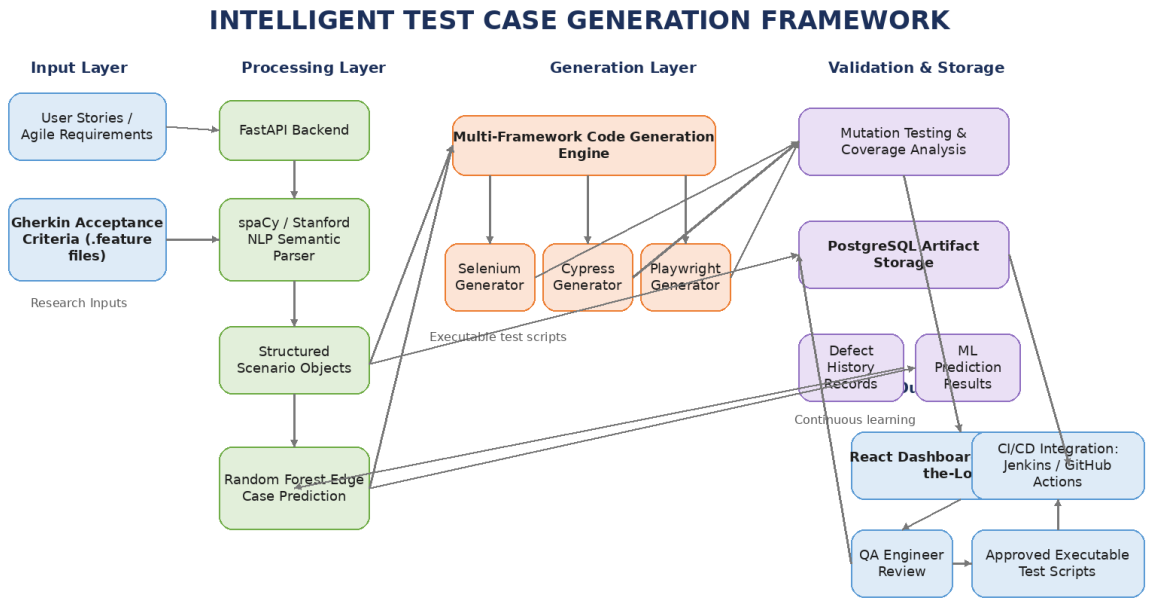


Figure: High-Level Architecture of the Intelligent Test Case Generation Framework

- High-Level System Architecture Diagram - Illustrating the end-to-end pipeline from Gherkin acceptance criteria input through the NLP semantic parsing engine, Random Forest edge case prediction module, multi-framework code generation engine, and React Dashboard output

- Component Interaction Diagram - Describing the interactions between the FastAPI backend, spaCy/Stanford NLP semantic parser, Random Forest ML classifier, Selenium/Cypress/Playwright code generators, mutation testing engine, and PostgreSQL artefact storage

- Data Flow Diagram for the Test Case Generation Module - Depicting how raw Gherkin .feature files are ingested, semantically parsed into structured scenario objects, enriched with ML-predicted edge cases, transformed into executable test scripts, and validated through mutation testing before being delivered to the QA engineer via the human-in-the-loop React Dashboard

- Entity Relationship Diagram (ERD) - Showing the database schema for storing Gherkin scenarios, generated test scripts, defect history records, ML prediction results, and mutation testing coverage reports

- CI/CD Integration Flow Diagram -Illustrating how the system integrates with Jenkins and GitHub Actions pipelines to trigger automated test generation upon new Gherkin file commits

Appendix C – Preliminary UI Mockups

- This appendix includes low-fidelity wireframes and preliminary UI mockups for the **React Dashboard**, which serves as the human-in-the-loop interface for QA engineers to review, approve, and export AI-generated test scenarios and scripts. Key screens include Failure analysis dashboard
    - Gherkin file upload and scenario preview panel
    - ML edge case prediction results and confidence score display
    - Generated test script review and approval interface (Selenium / Cypress / Playwright tabs)
    - Mutation testing coverage report and gap visualization
    - Test artefact export and CI/CD pipeline trigger panel

These mockups illustrate how developers and QA engineers will interact with the system.

Appendix D – Risk Analysis

Several risks may arise during the development and evaluation of the proposed system.

Risk	Likelihood	Impact	Mitigation Strategy
Gherkin parsing failures on non-standard syntax	Medium	High	Implement syntax validation and fallback parsing rules
Random Forest model underfitting on small defect datasets	Medium	High	Apply SMOTE oversampling and k-fold cross-validation
LLM-generated test scripts containing syntax errors	Medium	High	Integrate AST-based static analysis post-generation

Risk	Likelihood	Impact	Mitigation Strategy
CI/CD integration incompatibility	Low	Medium	Test against Jenkins, GitHub Actions, and GitLab CI
Insufficient real-world Gherkin dataset availability	Medium	High	Supplement with open-source BDD repositories and synthetic data

Appendix E – Ethical Considerations

All Gherkin acceptance criteria files, defect history datasets, and survey responses collected during this research will be handled in compliance with data privacy and research ethics standards. Specifically:

- All survey participants will provide **informed consent** before participating, and anonymity will be maintained throughout data analysis
- No proprietary source code, production credentials, or confidential business logic will be collected or stored during dataset collection
- All project-specific identifiers appearing in Gherkin input files will be **fully anonymized** before processing by the NLP pipeline
- Generated test artefacts used in evaluation experiments will be clearly labeled as AI-generated and will not be deployed in production environments without human review and approval